Python Class Test: Simple Answers

All 80 questions. Short answers in basic English with simple Python.

Section 1: The Way of the Program

1. What is a computer program?
A program is a list of steps for the computer. The computer does the steps one by one.

2. Source code vs output?
Source code is what you type. Output is what the computer shows after it runs.

3. Two types of errors?
Syntax error: you broke a Python rule. Runtime error: the code crashes while running.

4. Role of the interpreter?
It reads your Python and tells the computer what to do, line by line.

5. Display Welcome to Python.
Use print.

```
print("Welcome to Python")
```

Section 2: Variables, Expressions and Statements

1. What is a variable? A valid name?
A variable is a box with a name that holds a value. Example name: age.

```
age = 10
```

2. Output of x = 4; x = x + 3; print(x)?
The output is 7. We add 3 to 4 and save it back in x.

```
x = 4
x = x + 3
print(x)   # 7
```

3. Expression vs statement?
An expression makes a value (2 + 3). A statement does a job (x = 5).

4. Area of a rectangle?
Multiply length by width.

```
area = length * width
```

5. What is wrong with 2score?
A variable name cannot start with a number. Use score2 instead.

Section 3: Functions

1. What is a function?
A function is a named group of code. You write it once and use it many times.

2. Function greet that prints Hello.
Use def to make it.

```
def greet():
    print("Hello")
```

3. Purpose of parameters?
Parameters let you send information into a function.

```
def hi(name):
    print("Hi", name)
```

4. Defining vs calling?
Defining (def) makes the function. Calling runs it.

```
def hi():
    print("Hi")
hi()
```

5. Call double with 5.
Write the name and put 5 in the brackets.

```
double(5)
```

Section 4: Conditionals

1. Purpose of an if statement?
It lets the program choose. The code runs only if the condition is True.

2. Print Adult if age is at least 18.
Use >= for 'at least'.

```
if age >= 18:
    print("Adult")
```

3. elif vs else?
elif has its own condition. else has none and runs when nothing else was True.

4. 7 > 3 and 2 > 5?
False. With and, both sides must be True. 2 > 5 is False.

5. Print Even or Odd.
% gives the remainder. No remainder when dividing by 2 means even.

```
if n % 2 == 0:
    print("Even")
else:
    print("Odd")
```

Section 5: Recursion

1. What is recursion?
When a function calls itself.

2. What is a base case?
The simple case where the function stops and gives an answer.

3. No reachable base case?
The function never stops. Python gives a RecursionError.

4. Recursive factorial.
Stop at 0. Otherwise multiply n by factorial of n - 1.

```
def factorial(n):
    if n == 0:
        return 1
    return n * factorial(n - 1)
```

5. One difference: recursion vs iteration?
Recursion calls itself. Iteration uses a loop (for or while).

Section 6: Fruitful Functions

1. What is a fruitful function?
A function that gives back a value using return.

2. return vs print?
print only shows a value on screen. return gives the value back so you can use it.

3. Function square(n).
Return n times n.

```
def square(n):
    return n * n
```

4. Value of result = max(3, 9)?
result is 9, the bigger number.

5. Average of two numbers.
Add them and divide by 2.

```
def average(a, b):
    return (a + b) / 2
```

Section 7: Iteration

1. What is iteration?
Doing the same steps again and again, like in a loop.

2. Print 1 to 5.
range(1, 6) stops before 6.

```
for i in range(1, 6):
    print(i)
```

3. for vs while?
for repeats a set number of times. while repeats as long as a condition is True.

4. Purpose of break?
It stops the loop right away.

5. Count down from 3 to 1.
Subtract 1 each time.

```
n = 3
while n >= 1:
    print(n)
    n = n - 1
```

Section 8: Strings

1. What is a string?
Text inside quotes, like "hello".

2. Output of print("Python"[0])?
P. Counting starts at 0, so 0 is the first letter.

3. What is string slicing?
Cutting out a piece of a string. The end number is not included.

```
"Python"[0:3]   # "Pyt"
```

4. Convert "hello" to uppercase.
Use .upper().

```
print("hello".upper())   # HELLO
```

5. Result of len("coding")?
6, because coding has 6 letters.

Section 9: Lists

1. What is a list?
A group of items in order, inside [ ]. You can change it.

2. List of three languages.
Put them in square brackets.

```
langs = ["Python", "Java", "C"]
```

3. Second element of items?
Index 1, because counting starts at 0.

```
items[1]
```

4. append() vs remove()?
append adds an item at the end. remove takes an item out.

```
a = [1]
a.append(2)   # [1, 2]
a.remove(1)   # [2]
```

5. print([10, 20, 30][-1])?
30. -1 means the last item.

Section 10: Dictionaries

1. What is a dictionary?
A group of key and value pairs inside { }. You use the key to find the value.

2. Dictionary with name and age.
Use key: value pairs.

```
p = {"name": "Sam", "age": 10}
```

3. Get the value under name.
Put the key in square brackets.

```
p["name"]
```

4. Assign a new value to an existing key?
The old value is replaced by the new one.

```
p["age"] = 11
```

5. One difference from a list?
A list uses position numbers. A dictionary uses keys.

Section 11: Tuples

1. What is a tuple?
A group of items in order, inside ( ). It cannot be changed.

2. Tuple of 2, 4 and 6.
Use round brackets.

```
t = (2, 4, 6)
```

3. One difference: tuple vs list?
A tuple cannot be changed. A list can.

4. Output of print((5, 8, 11)[1])?
8. Index 1 is the second item.

5. Can you change a tuple element?
No. Python gives a TypeError. You must make a new tuple.

Section 12: Classes and Objects

1. What is a class?
A plan for making objects, like a blueprint.

2. What is an object?
A real thing made from a class.

3. Class Student, empty.
pass means do nothing.

```
class Student:
    pass
```

4. Class and instance?
The class is the plan. An instance is one thing made from it.

```
s1 = Student()
```

5. Purpose of __init__?
It runs when an object is made. It sets the starting values.

```
class Student:
    def __init__(self, name):
        self.name = name
```

Section 13: Classes and Functions

1. **How can a function create an object?**
   Call the class inside the function and return it.
2. **Function make_student().**
   Make a Student and return it.
   ```
   def make_student():
       return Student()
   ```
3. **Function outside vs inside a class?**
   Outside: a normal function. Inside: a method that belongs to the class.
4. **Why pass an object to a function?**
   So the function can read or change that object.
5. **Print an object's name.**
   Use a dot to get the attribute.
   ```
   def show(obj):
       print(obj.name)
   ```

Section 14: Classes and Methods

1. **What is a method?**
   A function inside a class. It is something the object can do.
2. **What does self refer to?**
   The object that is using the method.
3. **Class Dog with bark().**
   self goes first in the brackets.
   ```
   class Dog:
       def bark(self):
           print("Woof")
   ```
4. **Call bark on pet.**
   Use a dot.
   ```
   pet.bark()
   ```
5. **Instance attribute vs method?**
   An attribute is data the object has. A method is an action the object does.

Section 15: Inheritance

1. **What is inheritance?**
   A new class gets the things of an old class, like a child from a parent.
2. **Car inherits from Vehicle.**
   Put the parent name in brackets.
   ```
   class Car(Vehicle):
       pass
   ```
3. **One benefit of inheritance?**
   You reuse code, so you write less.
4. **What is method overriding?**
   A child class writes its own version of a parent's method.
   ```
   class Cat(Animal):
       def speak(self):
           print("Meow")
   ```
5. **Purpose of super()?**
   It lets the child use the parent's code.
   ```
   super().__init__()
   ```

Section 16: Analysis of Algorithms

1. **What is algorithm efficiency?**
   How fast an algorithm is and how little memory it uses.
2. **What is time complexity?**
   How the time grows when the data gets bigger.
3. **What does O(n) mean?**
   Time grows with the data. Double the data, double the work.
4. **O(n) or O(n^2) for large n?**
   O(n) is better. $O(n^2)$ gets much slower as n grows.
5. **Why does space complexity matter?**
   Computers have limited memory. Too much use can slow or crash a program.